IMPROVED SHARED SECRET ATTACKS IN NTRU AND THEIR RELATION WITH VFK LATTICES

Abstract.

1. Introduction

The NTRU cryptosystem was created by Hoffstein, Pipher, and Silverman in 1996 as a means of encryption and signature. Its security is based on the difficulty of solving a system of linear equations over polynomial rings, a problem that is expected to remain hard even with quantum computers. While it is not equivalent to the Shortest and Closest Vector Problems (SVP/CVP) in lattices, the NTRU cryptosystem has withstood over 20 years of cryptanalysis and variants of it have been shown to be secure under the Ring Learning With Error (R-LWE) hardness assumption. NTRU is known for its exceptional performance and moderate key-size, making it a popular choice for embedded cryptography. It has been standardized by IEEE, X9.98, PQCRYPTO, and was a finalist in the NIST post-quantum cryptography standardization effort. Ultimately, the Kyber algorithm was selected for standardization. It is also noted in the 3rd Status Report of NIST for PQC[1], the overall performance of any of these KEMs (i.e. Kyber, Saber and NTRU) would be acceptable for general-use applications. Also, NTRU-HRSS was chosen by Google for their ALTS protocol (ALTS:Application Layer Transport Security) in the hybrid model[2].

1.1. Relevant work.

The NTRU cryptosystem was first targeted for a lattice attack in 1997 by Coppersmith [10]. Later, Gentry developed an effective strategy, particularly useful when N is composite, see [15]. May, in [20], used his own class of lattices, called run-lattices, to solve similar issues.

Silverman [24] built on May's idea and proposed a technique that involves choosing r coefficients and reducing the dimension of the lattice to make them zero. A similar approach was taken by the authors of [14], who leveraged decryption failures to recover the secret key if the decryption oracle supported it.

Other techniques have been developed, such as reducing the NTRU problem to a multivariate quadratic system over a finite field with two elements using Witt vectors [5, 25]. Odlyzko [17] proposed a meet-in-the-middle attack that divides the search space into two parts, resulting in lower time complexity but high memory requirements. Howgrave's hybrid attack [16] combines lattice reduction with a meet-in-the-middle algorithm, and has been widely studied by researchers to evaluate the security of lattice-based encryption methods.

2010 *Mathematics Subject Classification.* 94A60.

Key words and phrases. Public Key Cryptography; NTRU Cryptosystem; Lattices; LLL algorithm; Closest Vector Problem; Voronoi First Kind;

[1] https://nvlpubs.nist.gov/nistpubs/ir/2022/NIST.IR.8413.pdf

[2] https://cloud.google.com/blog/products/identity-security/why-google-now-uses-post-quantum-cryptography-for-internal-comms

To attack the Ntru cryptosystem with a higher modulus than that in the NTRU-Encrypt standard, similar methods were proposed independently by Albrecht, Bai, and Ducas [2] and Cheon, Jeong, and Lee [6]. Kirchner [19] demonstrated that time complexity is polynomial for $q = 2^{\Omega(\sqrt{n \log\log n})}$ in $\mathbb{Q}(\zeta_{2^n})$. Lastly, Nguyen [23] enhanced and clarified the hybrid and meet-in-the-middle attacks. The subfield attack variation proposed in this paper is superior to the earlier methods, but not as effective as the hybrid attack.

1.2. **Our contribution.** Firstly, we replace the approximate CVP-oracle of [1] with an actual CVP-oracle, which allows for more precise and better results of our attack. Additionally, our attack is also extended to other NTRU variations such as NTRU-prime. Moreover, when we transfer our attack to VFK type NTRU lattices, we observe an improvement in the simplest Babai attack. This is particularly evident in `ntruhps2048509`, where our method is shown to be superior to Babai's attack in the same lattice. These advancements significantly enhances the previous attack [1]. What is more, our research has uncovered a non-trivial connection between NTRU lattice and a VFK lattice, which further enhances our understanding of these encryption systems.

1.3. **Roadmap.** Section 2 briefly covers integer lattices, while Section 4 provides the necessary background on Voronoi First Kind lattices. Moving on to Section 5, we delve into an explanation of the NTRU encryption system and its various variants.

2. Lattices

In this section, we recall some well-known facts about lattices which form the background to our algorithms.

Let $\mathbf{b}_1, \mathbf{b}_2, \ldots, \mathbf{b}_n$ linearly independent vectors of $\mathbb{R}^m$. The set

$$\mathcal{L} = \left\{ \sum_{j=1}^{n} \alpha_j \mathbf{b}_j : \alpha_j \in \mathbb{Z}, 1 \leq j \leq n \right\}$$

is called a *lattice* and the set $\mathcal{B} = \{\mathbf{b}_1, \ldots, \mathbf{b}_n\}$ a basis of $\mathcal{L}$. All the bases of $\mathcal{L}$ have the same number of elements n which is called *dimension* or *rank* of $\mathcal{L}$. If $n = m$, then the lattice $\mathcal{L}$ is said to have *full rank*. We denote by M the $n \times m$-matrix having as rows the vectors $\mathbf{b}_1, \ldots, \mathbf{b}_n$. If $\mathcal{L}$ has full rank, then the *volume* of the lattice $\mathcal{L}$ is defined to be the positive number $|\det M|$ which is independent from the basis $\mathcal{B}$. It is denoted by $vol(\mathcal{L})$ or $\det \mathcal{L}$ (see also [13]). If $\mathbf{v} \in \mathbb{R}^m$, then $\|\mathbf{v}\|$ denotes, as usually, the Euclidean norm of $\mathbf{v}$. We denote by $LLL(M)$, the application of well-known LLL-algorithm on the rows of M. Finally, we denote by $\lambda_1(\mathcal{L})$ the smallest of the lengths of vectors in $\mathcal{L} - \{\mathbf{0}\}$.

3. Closest Vector Problem

In this chapter we will define the Closest Vector Problem (CVP) [13] and its approximate variation which is the problem we will base our attack on the NTRU system. We will also provide Babai's algorithm [13] that solves the approximation CVP problem [13].

Definition 3.1. ***Closest Vector Problem*** *($CVP(\mathcal{L}, t)$): Given a lattice $\mathcal{L} \subset \mathbb{Z}^m$ of rank n and a vector $\boldsymbol{t} \in \mathbb{R}^m$, $\boldsymbol{t} \notin \mathcal{L}$, find a vector $\boldsymbol{u} \in \mathcal{L}$ such that, for every $\boldsymbol{v} \in \mathcal{L}$ we have:*

$$\|\boldsymbol{u} - \boldsymbol{t}\| \leq \|\boldsymbol{v} - \boldsymbol{t}\|$$

The closest vector problem is known to be NP-hard(Chapter 3 of [22]). We will now define the approximate-Closest Vector Problem ($CVP_{\gamma_n}(\mathcal{L}, \mathbf{t})$) as follows:

Definition 3.2. ***approximate-Closest Vector Problem*** *(($CVP_{\gamma_n}(\mathcal{L}, \mathbf{t})$) : Given a lattice $\mathcal{L} \subset \mathbb{Z}^m$ of rank n and a vector $\mathbf{t} \in \mathbb{R}^m$, find a vector $\mathbf{u} \in \mathcal{L}$ such that, for every $\mathbf{v} \in \mathcal{L}$ we have:*

$$\|\boldsymbol{u} - \boldsymbol{t}\| \leq \gamma_n \|\boldsymbol{v} - \boldsymbol{t}\| (\ for\ some\ \gamma_n \geq 1)$$

We say that we have a CVP oracle, if we have an efficient probabilistic algorithm that solves CVP_{γ_n} for $\gamma_n = 1$. To solve CVP_{γ_n}, we usually use Babai's algorithm [13, Chapter 18] (which has polynomial running time). In fact, combining this algorithm with LLL algorithm, we solve $\text{CVP}_{\gamma}(\mathcal{L}, \mathbf{t})$ for some lattice $\mathcal{L} \subset \mathbb{Z}^m$, for $\gamma_n = 2^{n/2}$ and $n = rank(\mathcal{L})$ in polynomial time. The pseudocode for Babai's algorithm is given in algorithm 1.

Algorithm 1: Babai's Nearest plane Algorithm

input : A $n \times m$-matrix M with rows the vectors of a basis $\mathcal{B} = \{\mathbf{b}_i\}_{1 \leq i \leq n}$ $\subset \mathbb{Z}^m$ of the lattice $\mathcal{L}$ and a vector $\mathbf{t} \in \text{span}(\mathcal{B})$

output: $\mathbf{x} \in L$ such that $||\mathbf{x} - \mathbf{t}|| \leq 2^{n/2} dist(L, \mathbf{t})$.

$M^* = \{(\mathbf{b}_j^*)_j\} \leftarrow GSO(M)$; `/* GSO: Gram-Schimdt Orthogonalization */`

$\mathbf{b} \leftarrow \mathbf{t}$;

for $j = n$ *to* 1 **do**

  $c_j \leftarrow \left\lfloor \frac{\mathbf{b} \cdot \mathbf{b}_j^*}{||\mathbf{b}_j^*||^2} \right\rceil$; `/*` $\lfloor x \rceil = \lfloor x + 0.5 \rfloor$ `*/`

  $\mathbf{b} \leftarrow \mathbf{b} - c_j \mathbf{b}_j$

end

return ***t-b***

4. Voronoi First Kind Lattices

First, we will define the Voronoi Cell and the relevant vectors of a Voronoi First Kind (VFK) Lattice [**?**] and afterwards we will express the criteria under which a lattice can be considered as VFK. Finally, we will describe an efficient algorithm [21] to solve the CVP problem for VFK Lattices.

Definition 4.1. ***Voronoi cell Vor($\mathcal{L}$)****: Given a lattice $\mathcal{L} \in \mathbb{R}^m$, the voronoi cell $Vor(\mathcal{L})$ is the set of points closer to the origin than to any other lattice point (with respect to the given norm):*

$$Vor(\mathcal{L}) = \{\boldsymbol{x} \in \mathbb{R}^n : \|\boldsymbol{x}\| \leq \|\boldsymbol{x} - \boldsymbol{v}\|\}, \forall \boldsymbol{v} \in \mathcal{L}$$

Vectors $\mathbf{v}$ for which there exists a point that is closer to $\mathbf{v}$ and the origin 0 than to any other lattice vector, are called **relevant vectors** and are denoted by $Rel(\mathbf{v})$. We are now ready to define the VFK Lattice.

Definition 4.2. ***Voronoi First Kind (VFK) Lattice****: A lattice $\mathcal{L}$ of dimension n is called Voronoi First Kind (VFK) if and only if there is a lattice basis $\{\mathbf{v}_i\}_{i=1,...,n}$ and a lattice vector $\mathbf{v}_0$ such that:*

(**i**). $\sum_{i=0}^{n} \mathbf{v}_i = \mathbf{0}$, *(superbasis condition)*

(**ii**) $q_{ij} = \mathbf{v}_i \cdot \mathbf{v}_j \leq 0$ *for* $i, j = 0, 1, ..., n$ *with* $i \neq j$. *(obtuse condition)*

The q_{ij} are called the selling parameters [27]. An important and useful property of VFK Lattices is the way its relevant vectors can be written as the sum of a set of its superbasis vectors:

$$\sum_{i \in I} \mathbf{b}_i \tag{4.1}$$

where I is a strict subset of $\{0, 1, .., n\}$ and $I \neq \emptyset$. We can now start explainig the main algorithm that we used for our attacks.

4.1. **CVP efficient algorithm for VFK lattices.** Let $\mathcal{L}$ be a VFK lattice in $\mathbb{R}^m$, with an obtuse superbasis $\{\mathbf{b}_i\}_{i=0,...,n}$ and $\mathbf{y}$ a point in $\mathbb{R}^m$. We also define B, a matrix whose columns are represented by the obtuse superbasis vectors, $\mathbf{y}$, a column vector such that $\mathbf{y} = B\mathbf{z}$ and $\mathbf{x}_0$ a lattice point in $\mathcal{L}$. In order to solve the CVP problem in VFK we need to find $n+1$ integers $\{w_i\}_{i=0,...,n}$ that minimize the norm:

$$\left\| \mathbf{y} - \sum_{i=0}^{n} \mathbf{b}_i w_i \right\|^2 = \left\| B(\mathbf{z} - w) \right\|^2 \tag{4.2}$$

Robby G. McKilliam, Alex Grant and I. Vaughan L. Clarkson proposed an interesting iterative procedure [21] that solves the CVP using the following steps:

Step 1: $\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{v}_k$

Step 2: $\mathbf{v}_k = argmin_{\mathbf{v} \in Rel(\mathcal{L}) \cup \{0\}} \|\mathbf{y} - \mathbf{x}_k - \mathbf{v}\|$

$\mathbf{v}_k$ is the relevant vector that minimizes the norm of Step 2, during the k-th iteration. There might be many relevant vectors that minimize this norm and the choice does not change the outcome of the algorithm. It is proven by the same authors [21] that after every iteration of the algorithm $\|\mathbf{y} - \mathbf{x}_k\| > \|\mathbf{y} - \mathbf{x}_{k+1}\|$ and that after at most n iterations of the algorithm, the closest vector point will be found. The choosing of $\mathbf{x}_0$ influences greatly the number of iterations the algorithm will do. Nevertheless, this part of the algorithm has a complexity of $\mathcal{O}(n)$. If we take into consideration the matrix B, the vector y and the property (4.2) then the iterative procedure becomes:

Step 1: $\mathbf{x}_{k+1} = B\mathbf{u}_{k+1}$

Step 2: $\mathbf{u}_{k+1} = \mathbf{u}_k + \mathbf{t}_k$

Step 3: $\mathbf{t}_k = argmin_{\mathbf{t}\in\{0,1\}^{n+1}} \|B(\mathbf{z} - \mathbf{u}_k - \mathbf{t})\|$

According to the property (4.1), a relevant vector $\mathbf{v}$ can be written as $\mathbf{v} = \sum_{i\in I} \mathbf{t}_i\mathbf{b}_i$, thus $\mathbf{t}_k$ represents the coefficients of a relevant vector, which is why its values are either 0 or 1.

The maximum number of relevant vectors that a lattice can have is $2^{n+1} - 2$, the complexity of which grows exponentialy, therefore another way to find the minimizations was found.

We set $p = \mathbf{z} - \mathbf{u}_k$ and the norm in (4.2) becomes:

$$\begin{aligned} \Big\|B(p - \mathbf{t})\Big\|^2 &= \Big\|\sum_{i=0}^{n} \mathbf{b}_i(p_i - \mathbf{t}_i)\Big\|^2 \\ &= \sum_{i=0}^{n}\sum_{j=0}^{n} q_{ij}p_ip_j - 2\sum_{i=0}^{n}\sum_{j=0}^{n} q_{ij}p_j\mathbf{t}_i + \sum_{i=0}^{n}\sum_{j=0}^{n} q_{ij}\mathbf{t}_i\mathbf{t}_j \end{aligned} \tag{4.3}$$

Let $s_i = -2\sum_{i=0}^{n}\sum_{j=0}^{n} q_{ij}p_j$ and the above quadratic form becomes: have:

$$Q(t) = \sum_{i=0}^{n} s_i\mathbf{t}_i + \sum_{i=0}^{n}\sum_{j=0}^{n} q_{ij}\mathbf{t}_i\mathbf{t}_j \tag{4.4}$$

Robby G. McKilliam, Alex Grant and I. Vaughan L. Clarkson have shown [21] that a minimizer can be found in Q(t) by ccomputing a minimum cut in un undirected flow network, since there is an one-to-one correspondence between the obtuse superbasis of the VFK lattice and the Laplacian matrix.

Definition 4.3. *An undirected flow network is a graph G with $n+3$ vertices v_i and edges e_{ij} connecting v_i to v_j with weight w_{ij}. The graph is undirected, therefore $w_{ij} = w_{ji}$. The first vertice v_0 is called source and the last one, v_{n+2}, is the sink.*

Definition 4.4. *A cut is a subset $C \subset V$, where V is the set of vertices in a graph, and its complement $\bar{C} \subset V$ and $v_0 \in C, v_{n+2} \in \bar{C}$.*

Definition 4.5. *The weight W of a cut is the sum of the weights on the edges crossing from the vertices in C to the vertices in $\bar{C}$:*

$$W = \sum_{i\in I}\sum_{j\in J} w_{ij}$$

Where $I = \{i|v_i \in C\}, J = \{j|v_j \in \bar{C}\}$. The weight w_{ii} of the edge e_{ii} does not change the value of W.

Definition 4.6. *Mincut is a C and a $\bar{C}$ that minimize the weight W.*

It was shown in [21] that the weight W can be represented with the following quadratic form:

$$
\begin{aligned}
F(t) &= \sum_{i=0}^{n} g_i \ \mathbf{t}_i - \sum_{i=0}^{n}\sum_{j=0}^{n} w_{ij}\mathbf{t}_i\mathbf{t}_j \\
g_i &= d_i + \sum_{j=0}^{n} w_{ij} \\
d_i &= w_{i,n+2} - w_{0,i}
\end{aligned} \tag{4.5}
$$

From the equivalence of (4.4) and (4.5) we calculate the weigths of our graph:

$$
\begin{aligned}
w_{i,j} &= -q_{ij} \ (selling \ parameters) \\
s_i &= d_i + \sum_{j=0}^{n} w_{ij}
\end{aligned} \tag{4.6}
$$

$$
\begin{aligned}
w_{0,i} = 0, \ if \ s_i \geq 0, \ else \ w_{0,i} = -s_i \\
w_{i,n+2} = s_i, \ if \ s_i \geq 0, \ else \ w_{i,n+2} = 0
\end{aligned} \tag{4.7}
$$

We create vector $\mathbf{t}$ from set C like this:

$$
\mathbf{t}_i = 1, \ if \ i \in C, \ else \ \mathbf{t}_i = 0 \tag{4.8}
$$

From the above we deduct that if we do a minimum cut in a graph G with $n+3$ vertices and the choose the weights of the edges according to (4.6) and (4.8), then the vertices that exist in C can create the vector t of Step 3 of the iterative procedure. This minimization is computed in $\mathcal{O}(n^3)$ operations. The total complexity of the algorithm is $\mathcal{O}(n) * \mathcal{O}(n^3) = \mathcal{O}(n^4)$.

The pseudocode of the algorithm that we used for our experiments can be found at the Appendix A.

5. NTRU

Let the polynomial ring $\mathcal{R} = \mathbb{Z}[x]/\langle D(x)\rangle$ for some $D(x) \in \mathbb{Z}[x]$ and $\langle D(x)\rangle$ the ideal generated by $D(x)$. We write $*$ for the multiplication in the ring. Also, fix a polynomial $h(x) \in \mathbb{Z}[x]$ of degree N. We set,

$$
B_h = \begin{pmatrix} - \ h(x) \ - \\ - \ x * h(x) \ - \\ \vdots \\ - \ x^{N-1} * h(x) \ - \end{pmatrix}, \tag{5.1}
$$

where with $x^i * h(x)$ we mean the vector with coordinates the coefficients of the polynomial after multiplication in $\mathcal{R}$. When we write the coefficient vector of $h(x) = a_{N-1}x^{N-1} + \cdots + a_0$ we usually denoted with $\mathbf{h} = (a_0, ..., a_{N-1})$. Then, multiplication $g(x) * h(x)$ in $\mathcal{R}$ can be represented as the multiplication of the row matrix $[\mathbf{g}]$ and matrix B_h, i.e. $[\mathbf{g}]B_h$.

Examples. (i). Let $D(x) = x^3 + 1$ and $h(x) = x + 1$. Then, the multiplication

$(x^2+1)*h(x) = x^2+x$ in $\mathcal{R}$ can be written as

$$[1,0,1]B_h = [1,0,1]\begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ -1 & 0 & 1 \end{bmatrix} = [0,1,1] \to x + x^2.$$

(ii). If $D(x) = x^N - 1$, then the matrix B_h for some polynomial $h(x)$ in $\mathcal{R}$ is the cyclic matrix generated by the coefficients of $h(x)$.
(iii). If $D(x) = x^3 - x - 1$ and $h(x) = x + 1$ then $(x^2+1)*h(x) = x^2 + 2x + 2$, and

$$[1,0,1]B_h = [1,0,1]\begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = [2,2,1] \to 2 + 2x + x^2.$$

The set

$$\mathcal{L}_h = \{(f(x), g(x)) \in \mathcal{R}^2 : g(x) = f(x) * h(x)\}$$

where h has degree say N, is a lattice. Indeed,

$$\mathcal{L}_h = \mathbb{Z}^{2N} B'_h$$

where

$$B'_h = \left[\frac{B_h}{qI_N}\right].$$

If you consider the previous lattice, but taking $\mod q$ (for some positive q) we get a (NTRU type) lattice

$$\mathcal{L}_h^q = \{(f(x), g(x)) \in \mathcal{R}^2 : g(x) = f(x) * h(x) \pmod q\}$$

where we also write it as,

$$\mathcal{L}_h^q = \{(\mathbf{f}, \mathbf{g}) \in \mathbb{Z}^{2N} : [\mathbf{g}] = [\mathbf{f}]M_h\}$$

where

$$M_h = \left[\begin{array}{c|c} I_N & B_h \\ \hline \mathbf{0}_N & qI_N \end{array}\right].$$

The lattice $\mathcal{L}_h$ has several interesting properties when B_h is a cyclic matrix (see (5.1)). One example is when we choose $h(x) = x^N - 1$. In this case, if $(\mathbf{a}, \mathbf{b})$ is a vector in the lattice, then performing a cyclic permutation of $\mathbf{a}$ and $\mathbf{b}$ $k-$times will result in another vector in the lattice.

5.1. **NTRU.** We first define a Key Encapsulation Mechanism (KEM) which consists from three algorithms.
keyGen. A probabilistic algorithm that outputs a public key(pk) and a secret key(sk).
Encap. A probabilistic algorithm that takes as input the public key and outputs a ciphertext(c) and a shared secret(k).
Decap. A probabilistic algorithm that takes as input the ciphertext and the secret key, and outputs the shared secret.

Let q, N be positive integers with N prime. A ternary polynomial is one that has as coefficients only the integers $-1, 0, 1$. With $\mathcal{T}_a$ we denote the set of ternary polynomials of $\mathcal{R}$ with degree at most a and $\mathcal{T}_a(d_1, d_2) \subset \mathcal{T}_a$ consists from elements of $\mathcal{T}_a$ with d_1 coefficients equal to 1 and d_2 equal to -1. Furthermore, with $\mathcal{T}_a(w), w \in \mathbb{Z}_{>0}$ we denote the ternary polynomials which have w non-zero coefficients. With $\mathcal{T}_{a,+}$ we denote the subset of $\mathcal{T}_a$ consisting from polynomials

$f(x) = \sum_{i=0}^{\alpha} v_i x^i$ such that $\sum_{i=0}^{a-1} v_i v_{i+1} \geq 0$. Let $D(x)$ be a polynomial of degree N. We also have four sample spaces, $\mathcal{L}_f, \mathcal{L}_g, \mathcal{L}_r$, and $\mathcal{L}_m$. We set, $\Phi_1(x) = x - 1$ and in the case of $D(x) = x^N - 1$, we set $\Phi_N(x) = D(x)/\Phi_1(x)$. What is more, we set

– $\mathcal{R} = \mathbb{Z}[x]/\langle D(x)\rangle$, $\mathcal{R}/3 = \mathbb{Z}_3[x]/\langle D(x)\rangle$ and $\mathcal{R}/q = \mathbb{Z}_q[x]/\langle D(x)\rangle$.

– $\mathcal{S} = \mathbb{Z}[x]/\langle \Phi_N(x)\rangle$, $\mathcal{S}/3 = \mathbb{Z}_3[x]/\langle \Phi_N(x)\rangle$ and $\mathcal{S}/q = \mathbb{Z}_q[x]/\langle \Phi_N(x)\rangle$.

5.2. **Generation of Public key.** We choose the *secret key*, $(g(x), f(x))$ from $\mathcal{L}_g$ and $\mathcal{L}_f$, respectively, which both are suitable subsets of $\mathcal{T}_a$ (for some a). Also $g(x)$ is invertible in $\mathcal{R}/3$. The *public key* is $h(x) = g(x)/(\varepsilon f(x))$ invertible in $\mathcal{R}/q$, where $\varepsilon = 1/3$ or 3. Since there are many different sample spaces in the bibliography, here for NTRU-HPS and NTRU-HRSS, we follow [7] and for NTRU-prime we follow [3]. In all the flavors we ignore encoding/decoding operations.

5.3. **Sample spaces.**

5.3.1. *NTRU-HPS.* N is prime, and $q \leq 16N/3 + 16$ which is a power of two, $\varepsilon = 1/3$, $D(x) = x^N - 1$.
– $\mathcal{L}_m = \mathcal{L}_g = \mathcal{T}_{N-2}(\frac{q}{8} - 2, \frac{q}{8} - 2)$,
– $\mathcal{L}_f = \mathcal{L}_r = \mathcal{T}_{N-2}$.

5.3.2. *NTRU-HRSS.* N is prime and $q = 2^{\lceil 7/2 + \log_2 N\rceil} > 8\sqrt{2}(N-1)$, $\varepsilon = 1/3$, $D(x) = x^N - 1$,
– $\mathcal{L}_m = \mathcal{L}_r = \mathcal{T}_{N-2}$,
– $\mathcal{L}_f = \mathcal{T}_{N-2,+}$,
– $\mathcal{L}_g = \{(x-1) * v(x) : v(x)\ in\ \mathcal{T}_{N-2,+}\}$.
Remark that NTRU-HRSS uses arbitrary weight sample spaces.

5.3.3. *NTRU-prime.* NTRU-prime is different from the two previous flavors, since in this case $\mathcal{R}$ is a field, whilst in the previous cases, it was a ring but not a field (see [8]). We set $\varepsilon = 3$ and let w, p, q positive integers with p, q primes and such that $w \geq 1, p \geq 1.5w, q \geq 16w + 1$. We set $D(x) = x^p - x - 1$, which is irreducible mod q.
– $\mathcal{L}_g = \mathcal{T}_{p-1}$, $\mathcal{L}_f = \mathcal{L}_r = \mathcal{T}_{p-1}(w)$.
Here we do not use $\mathcal{L}_m$, but this set implicitly defined. We choose randomly $g(x)$ from $\mathcal{L}_g$ invertible in $\mathcal{R}/3$, and $f(x)$ randomly from $\mathcal{L}_f$. The public key is $h(x) = g(x)/(3f(x))$ in $\mathcal{R}/q$. Remark that, $w > 0$ and $\mathcal{R}/q$ is a field, thus polynomial $h(x)$ is invertible in $\mathcal{R}/q$.

5.4. **Encap/Decap for NTRU-HRSS/HPS.** We provide Encapsulation/ Decapsulation algorithms for NTRU-HPS and NTRU-HRSS. The algorithm for these two flavors are the same and only the sample spaces are different. We define the function $\text{Lift}_3 : \mathcal{L}_m \to \mathcal{S}/3$ be the reduction in the ring $\mathcal{S}/3$ i.e the unique polynomial of degree at most $N-2$ with coefficients $\{-1, 0, 1\}$. For the HRSS the analogous function is $\text{Lift}_3' : \mathcal{L}_m \to \mathcal{S}/3$ such that $\text{Lift}_3'(a(x)) = (x-1) * \text{Lift}_3(a(x)/(x-1))$. For a pseudocode that implements Lift_3', see [18, Appendix 2 - Algorithm 8].

GenKey. Takes as input a *seed* and outputs a quadruple $(\mathbf{h}, (\mathbf{f}, \mathbf{f}_3, \mathbf{h}_q))$.

`1.` $(f(x), g(x)) \xleftarrow{\$} \mathcal{L}_f \times \mathcal{L}_g$ `# here we use the seed`

2. $f_q(x) \leftarrow f^{-1}(x) \mod (q, \Phi_N(x))$
3. $f_3(x) \leftarrow f^{-1}(x) \mod (3, \Phi_N(x))$
4. $h(x) \leftarrow 3g(x) * f_q(x) \mod (q, D(x))$ $\#D(x) = x^N - 1$
5. $h_q(x) \leftarrow h^{-1}(x) \mod (q, \Phi_N(x))$
6. $\mathbb{S} \xleftarrow{\$} \{0,1\}^{256}$
7. **return** $(pk, sk) = (\mathbf{h}, (\mathbf{f}, \mathbf{f}_3, \mathbf{h}_q, \mathbb{S}))$

Encapsulation. Takes as input the pk and outputs a pair, ciphertext c and shared secret s.

1. $(r(x), m(x)) \xleftarrow{\$} \mathcal{L}_r \times \mathcal{L}_m$
2. $c(x) \leftarrow h(x) * r(x) + \text{Lift}_3(m(x)) \mod (q, \Phi_1\Phi_N)$ (or $\text{Lift}'_3(m(x))$ for HRSS)
3. $s \leftarrow \text{hash}(\mathbf{r}, \mathbf{m})$
4. **return** $(\mathbf{c}, s)$

Decapsulation. It takes as input the (sk, $\mathcal{S}$) and the ciphertext, and outputs the shared secret s.

1. $(\mathbf{m}, \mathbf{r}) \leftarrow \text{Decrypt}(sk, \mathbf{c})$
2. $s \leftarrow \text{hash}(\mathbf{r}, \mathbf{m})$
3. **if** fail $= 0$ **then**
4. **return** s
5. **else**
6. **return** $hash(\mathcal{S}, \mathbf{c})$

fail is a boolean function that accepts $(\mathbf{c}, \mathbf{r}, \mathbf{m})$, and returns 0 if the decryption is correct. The function *Decrypt* is the following. +++++

5.5. **Encap/Decap for NTRU-Prime.** We need the following sets,

$$A_{q,1} = \{-(q-1)/2, ..., -6, -3, 0, 3, 6, ..., (q-1)/2\},$$
$$A_{q,2} = \{-(q+1)/2, ..., -6, -3, 0, 3, 6, ..., (q+1)/2\}$$

and `Round_set` is the set of all polynomials

$$\sum_{j=0}^{p-1} r_j x^j \in \mathcal{R} \text{ with } r_j \in A_{q,1} \text{ if } q \equiv 1 \pmod 3,$$

else $r_j \in A_{q,2}$. Also, we consider a function `closest`:$\mathbb{Z} \to 3\mathbb{Z}$, where `closest`$(x)$ is the multiple of 3 nearest to x. I.e. is defined

$$\texttt{closest}(x) = \begin{cases} x\lceil x/3 \rceil & x \geq 0 \\ x\lfloor x/3 \rfloor & x < 0 \end{cases}$$

Now, the function `round` : $\mathcal{R}/q \to$ `Round_set` is such that `round` = `closest` $\circ \text{Lift}_q$. Where $\text{Lift}_q : \mathcal{L}_m \to \mathcal{R}/q$.

GenKey. Takes as input a *seed* and outputs a triple $(\mathbf{h}, (\mathbf{f}, \mathbf{g}_3))$.

1. $f(x) \xleftarrow{\$} \mathcal{L}_f$ # `here` L_f `is` $\mathcal{T}_{p-1}(w)$
2. $g(x) \xleftarrow{\$} \mathcal{L}_g$ # `with` $g(x)$ `invertible modulo` 3. `Also` L_g `is` $\mathcal{T}_{p-1}$

3. $g_3(x) \leftarrow g^{-1}(x) \mod (3, D(x))$
4. $f_q(x) \leftarrow f^{-1}(x) \mod (q, D(x))$
5. $h(x) \leftarrow g(x) * f_q(x)/3 \mod (q, D(x))$
6. **return** $(pk, sk) = (\mathbf{h}, (\mathbf{f}, \mathbf{g}_3))$

Encapsulation.
It takes as input a seed and the public key and outputs a ciphertext and a shared key.

1. $r(x) \xleftarrow{\$} \mathcal{L}_r$ # here L_r is $\mathcal{T}_{p-1}(w)$
2. $c_1(x) \leftarrow h(x) * r(x) \mod (q, D(x))$
3. $c(x) \leftarrow \texttt{round}(c_1(x))$
4. $\mathbf{C} \leftarrow (\mathbf{c}, Hash(\mathbf{r}, \mathbf{h}))$ #ciphertext
5. $\mathbf{K} \leftarrow Hash(1, \mathbf{r}, \mathbf{C})$ # shared key
6. **return** $(\mathbf{C}, \mathbf{K})$

Decapsulation. Input $\mathbf{C}$ and the public, secret key.
1. $e'(x) \leftarrow 3f(x) * c(x) \mod (q, D(x))$
2. $e''(x) \leftarrow Lift_q(e'(x))$
3. $e(x) \leftarrow e''(x) \mod 3$
4. $r''(x) \leftarrow e(x) * g_3(x) \mod 3$
5. $r'(x) \leftarrow Lift_3(r''(x))$
6. **If** $weight(r'(x)) \neq w$, **then**
7. **return** $(1, 1, ..., 1; 0, 0, ..., 0)$
8. $c'(x) \leftarrow \texttt{round}(h(x) * r'(x))$
9. $\mathbf{C}' \leftarrow (\mathbf{c}', Hash(\mathbf{r}', \mathbf{h}))$
10. **If** $\mathbf{C}' = \mathbf{C}$ **then**
11. **return** $Hash(1, \mathbf{r}', \mathbf{h})$

6. Auxiliary Results

Let k, N and q be positive integers. We set

$$M_k = \left[\begin{array}{c|c} I_N & -kI_N \\ \hline \mathbf{0}_N & qI_N \end{array}\right].$$

Let $\mathcal{L}_{k,q,N}$ be the lattice generated by the rows of M_k. We write $\mathcal{L}_k$ for the previous lattice.

Before we present the Proposition we remark that if $k = q$ the lattice is VFK. Indeed, we can check that the rows produce an obtuse superbasis,

$$\mathbf{v}_i \cdot \mathbf{v}_j = 0 \text{ and } \mathbf{v}_i \cdot \mathbf{v}_{N+j} \leq 0, \ (1 \leq i < j \leq N),$$

and $\mathbf{v}_0 = -\mathbf{v}_1 - \cdots - \mathbf{v}_{2N} = (-1, -1, ..., -1; 0, 0, ..., 0)$, is such that

$$\mathbf{v}_0 \cdot \mathbf{v}_j \leq 0 \ \ (j = 1, 2, ..., 2N).$$

Proposition 6.1. *If $k \leq \sqrt{2q-1}$, then the lattice $\mathcal{L}_k$ is VFK.*

Proof. We consider the unimodular matrix $U_P \in GL_{2N}(\mathbb{Z})$ (for some integer P),

$$U_P = \left[\begin{array}{c|c} I_N & \mathbf{0}_N \\ \hline PI_N & I_N \end{array}\right] = \begin{bmatrix} 1 & 0 & \dots & 0 & 0 & 0 & \dots & 0 \\ 0 & 1 & \dots & 0 & 0 & 0 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & 1 & 0 & 0 & \dots & 0 \\ P & 0 & \dots & 0 & 1 & 0 & \dots & 0 \\ 0 & P & \dots & 0 & 0 & 1 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & P & 0 & 0 & \dots & 1 \end{bmatrix}. \tag{6.1}$$

We shall prove that we can find an integer P such that, the rows of the matrix $U_P M_k$ is an obtuse superbasis of $\mathcal{L}$. This integer P will depend on k and q,

We first compute the matrix

$$A = U_P M_k = \left[\begin{array}{c|c} I_N & -kI_N \\ \hline PI_N & (q-Pk)I_N \end{array}\right]. \tag{6.2}$$

The superbasis is

$$\begin{array}{rcl} \mathbf{v}_1 & = & (1,0,...,0;-k,0,...,0) \\ \mathbf{v}_2 & = & (0,1,...,0;0,-k,...,0) \\ ... & & \\ \mathbf{v}_N & = & (0,0,...,1;0,0,...,-k) \\ \mathbf{v}_{N+1} & = & (P,0,...,0;q-Pk,0,...,0) \\ ... & & \\ \mathbf{v}_{2N} & = & (0,0,...,P;0,0,...,q-Pk) \end{array}$$

and

$$\mathbf{v}_0 = -\sum_{j=1}^{2N} \mathbf{v}_j = (-1-P,...,-1-P;kP+k-q,...,kP+k-q).$$

Since $\mathbf{v}_i$ and $\mathbf{v}_{i+N}$ have non zero elements only at positions $i, N+i$ we conclude that

$$\mathbf{v}_i \cdot \mathbf{v}_{j+N} = 0, \ (i \neq j, \ 1 \leq i,j \leq N).$$

Further,

$$\mathbf{v}_1 \cdot \mathbf{v}_{N+1} = \cdots = \mathbf{v}_i \cdot \mathbf{v}_{i+N} = \cdots = (kP-q)k + P, \ (1 \leq i \leq N). \tag{6.3}$$

Similarly,

$$\mathbf{v}_i \cdot \mathbf{v}_j = 0, \quad \mathbf{v}_{i+N} \cdot \mathbf{v}_{j+N} = 0, \ (1 \leq i < j \leq N),$$

$$\mathbf{v}_0 \cdot \mathbf{v}_i = -(kP+k-q)k - P - 1, \ (1 \leq i \leq N), \tag{6.4}$$

and

$$\mathbf{v}_0 \cdot \mathbf{v}_{i+N} = (kP+k-q)(q-kP) - (P+1)P, \quad (1 \leq i \leq N). \tag{6.5}$$

From (6.3), in order to have $\mathbf{v}_i \cdot \mathbf{v}_{i+N} \leq 0 \ \ (1 \leq i \leq n)$, we get

$$P \leq \beta = \frac{kq}{k^2+1}$$

and similarly from (6.4) we get

$$P \geq \alpha = \frac{kq - k^2 - 1}{k^2 + 1}.$$

Remark that, $\beta - \alpha = 1$. So, there is at least one integer value in $[\alpha, \beta]$, the value

$$P = \lfloor kq/(k^2+1) \rfloor. \tag{6.6}$$

Assume that $\beta \notin \mathbb{Z}$. Else, $P = \beta$. Consider the polynomial,

$$g(x) = -x^2(1+k^2) + x(-k^2 + 2kq - 1) + q(k-q).$$

Therefore, it is enough the relation (6.5) be non positive at P, i.e. $g(P) \leq 0$. But the discriminant of $g(x)$ equals to $\Delta = (k^2+1)^2 - 4q^2$. So, to have $g(x) \leq 0$ it is enough $\Delta \leq 0$, therefore we conclude $(1+k^2)^2 \leq 4q^2$. The result follows. □

Remark 6.1. *We set* $r = -[(kP + k - q)k + P + 1]$ *and*

$$s = (kP + k - q)(q - kP) - (P+1)P.$$

The Selling parameters $q_{ij} = \mathbf{v}_i \cdot \mathbf{v}_j$, $0 \leq i, j \leq 2N$ *are given by the symmetric* $(2N+1) \times (2N+1)$ *matrix,* $Q = (q_{ij}) =$

$$= \left[\begin{array}{c|c|c} N[(1+P)^2 + (k(P+1) - q)^2] & r\mathbf{1}_{1\times N} & s\mathbf{1}_{1\times N} \\ \hline r\mathbf{1}_{N\times 1} & (1+k^2)I_N & \big((kP-q)k + P\big)I_N \\ \hline s\mathbf{1}_{N\times 1} & \big((kP-q)k+P\big)I_N & \big((P^2 + (q - Pk)^2\big)I_N \end{array}\right].$$

where $\mathbf{1}_{1\times N}$ *is a row with* N *ones and* $\mathbf{1}_{N\times 1}$ *a column with* N *ones.*

Remark 6.2. *The maximum* k *such that* $g(P) < 0$ *for various* q *:*

q	*32*	*64*	*128*	*256*	*512*	*1024*	*2048*
k	*8*	*11*	*16*	*23*	*32*	*47*	*64*
P	*3*	*5*	*7*	*11*	*15*	*21*	*31*

7. Attack

Idea. For NTRU-prime we set $N = p$ and $m(x) = \texttt{round}(h(x) * r(x)) - h(x) * r(x)$. In $\mathcal{R}/q$ we have $c(x) = r(x) * h(x) + m(x)$. Multiplying both sides with $-k$ we get $km(x) = b(x) + u(x)$, where $u(x) = -kr(x) * h(x)$ and $b(x) = kc(x)$. We set $\mathbf{V}$ be the unknown vector $(\mathbf{m}, \mathbf{u})$. Assume that we can find a vector $\mathbf{E} \in \mathbb{Z}^{2N}$ such that,

$$||\mathbf{V} - \mathbf{E}|| < \frac{1}{2}\lambda_1.$$

We set

$$\mathbf{t} = (E_1, ..., E_N, b_1 + E_{N+1}, ..., b_N + E_{2N}).$$

We remark that $(\mathbf{m}, \mathbf{b} + \mathbf{u})$ is in $\mathcal{L}_k$. Indeed,

$$(\mathbf{m}, \mathbf{v})M_k = (\mathbf{m}, \mathbf{v}) \left[\begin{array}{c|c} I_N & -kI_N \\ \hline \mathbf{0}_N & qI_N \end{array}\right] = (\mathbf{m}, -k\mathbf{m} + q\mathbf{v}) = (\mathbf{m}, \mathbf{b} + \mathbf{u}).$$

Then, $\mathbf{W} \leftarrow BDD(\mathcal{L}_k, \mathbf{t})$ provides the message $\mathbf{m}$. Indeed,

$$||\mathbf{W} - \mathbf{t}|| \leq ||(\mathbf{m}, \mathbf{b} + \mathbf{u}) - \mathbf{t}|| = ||\mathbf{V} - \mathbf{E}|| < \frac{1}{2}\lambda_1.$$

Finally,

$$||\mathbf{W} - (\mathbf{m}, \mathbf{b} + \mathbf{u})|| \leq ||\mathbf{W} - \mathbf{t}|| + ||(\mathbf{m}, \mathbf{b} + \mathbf{u}) - \mathbf{t}|| < \lambda_1.$$

But, $\mathbf{W} - (\mathbf{m}, \mathbf{b} + \mathbf{u}) \in \mathcal{L}_k$ thus, $\mathbf{W} = (\mathbf{m}, \mathbf{b} + \mathbf{u})$. We conclude therefore that the first $N-$coordinates of $\mathbf{W}$ provide the message $\mathbf{m}$.

8. Experiments

8.1. **NTRU-HPS.** For the example 1 of [1], we used $(N, p, q, d) = (239, 3, 2^8, 71)$ and $y = 2.3$, we get an improvement. In [1] we have a successful attack for range $R = 9$, where we assumed that we know an approximation $\mathbf{E} = (\mathbf{0}_N, \mathbf{E}'_i)$, such that $|E'_i - c_i| \leq R$. With our enhanced method, using a CVP oracle instead of an approximation CVP oracle, we get $R = 11$. Furthermore, if we use the VFK type matrix used in CVP but using Babai's approximation algorithm we also get similar results. It seems that Babai behaves very well in VFK lattices. See also Fig.3, which tell us for the previous parameters the closest vector and the output of Babai's algorithm is very close.

For our next experiment, we utilized the parameters $(N, p, q, d) = (509, 3, 2048, 10)$. This example, also mentioned in [1, example 5], is especially noteworthy as it pertains to the `ntruhps2048509`. According to [1], the maximum range R for this case is limited to 26. Our new attack has considerably improved this limit to $R = 32$. It is worth noting that, in this instance, the min-cut algorithm proved successful and Babai was able to achieve a range of $R = 31$.

The last experiment uses $(N, p, q, d) = (667, 3, 2048, 10)$. The same experiment in [1] provides $R = 19$ and our new attack provides $R = 32$.

8.1.1. *Further Experiments.* In the previous subsection we used unweighted messages. If we choose weight messages then our attack is better, since we can have a better guessing for $\mathbf{m}$.

8.2. **NTRU-HRSS.**

8.3. **NTRU-prime.**

9. Conclusion

References

[1] M. Adamoudis and K. A. Draziotis, Message recovery attack to NTRU using a lattice independent from the public key, arXiv:2203.09620, 2022, url : `https://arxiv.org/abs/2203.09620`

[2] M. Albrecht, S. Bai, and L. Ducas, A subfield lattice attack on overstretched NTRU assumptions. CRYPTO 2016. LNCS **9814**, Springer 2016.

[3] D. G. Bernstein, C. Chuengsatiansup, T. Lange, C. van Vredendaal. NTRU Prime: Reducing Attack Surface at Low Cost. In: Adams, C., Camenisch, J. (eds) Selected Areas in Cryptography – SAC 2017. SAC 2017. Lecture Notes in Computer Science(), vol 10719. Springer, Cham. https://doi.org/10.1007/978-3-319-72565-9_12. 2018.

[4] Daniel J. Bernstein, Billy Bob Brumley, Ming-Shing Chen, Chitchanok Chuengsatiansup, Tanja Lange, Adrian Marotzke, Bo-Yuan Peng, Nicola Tuveri, Christine van Vredendaal, Bo-Yin Yang, NTRU - Prime Algorithm Specifications And Supporting Documentation, 2020.

[5] Gerald Bourgeois and Jean-Charles Faugère, Algebraic attack on NTRU using Witt vectors and Gröbner bases, Journal of Mathematical Cryptology **3(3)** p. 205–214, 2009.

[6] J. H. Cheon, J. Jeong, and C. Lee, An algorithm for NTRU problems and cryptanalysis of the GGH multilinear map without an encoding of zero. Cryptology ePrint Archive, Report 2016/139, 2016.

[7] C. Chen, O. Danba, J. Hofstein, A. Hülsing, J. Rijneveld, J. M. Schanck, T. Saito, P. Schwabe, W. Whyte, K. Xagawa, T. Yamakawa, Z. Zhang, NTRU - Algorithm Specifications And Supporting Documentation, 2020.

[8] Hao Cheng, Daniel Dinu, Johann Grobschadl, Peter B. Ronne, and Peter Y. A. Ryan, A Lightweight Implementation of NTRU Prime for the Post-Quantum Internet of Things, https://hcheng.cc/publication/wistp2019/WISTP2019.pdf

[9] Conway, J. Sloane, N., Low-Dimensional Lattices VI: Voronoi Reduction of Three-Dimensional Lattices. Proceedings of The Royal Society A Mathematical Physical and Engineering Sciences. 436. 10.1098/rspa.1992.0004, 2001

[10] D. Coppersmith and A. Shamir, Lattice Attacks on NTRU. In Proc. Eurocrypt 1997, LNCS **1223**.

[11] Dantzig, G.B.; Fulkerson, D.R., On the max-flow min-cut theorem of networks, url: https://web.archive.org/web/20180505093256/http://www.dtic.mil/dtic/tr/fulltext/u2/605014.pdf, 1964

[12] Ford, L., Fulkerson, D., Maximal Flow Through a Network. Canadian Journal of Mathematics, 8, 399-404. doi:10.4153/CJM-1956-045-5, 1956

[13] S. Galbraith, Mathematics of Public key Cryptography, Cambridge university press, 2012.

[14] N. Gama and Phong Q. Nguyen, New Chosen-Ciphertext Attacks on NTRU. Public Key Cryptography – PKC 2007, LNCS **4450**, Springer 2007.

[15] C. Gentry, Key recovery and message attacks on NTRU-composite, EUROCRYPT 2001, LNCS **2045**, Springer 2001.

[16] N. Howgrave-Graham. A Hybrid Lattice-Reduction and Meet-in-the-Middle Attack Against NTRU. CRYPTO 2007, LNCS **4622**, Springer 2007.

[17] N. Howgrave-Graham, J. H. Silverman, and W. Whyte, Meet-in-the-middle Attack on an NTRU private key, Technical report, NTRU Cryptosystems, July 2006. Report 04, available at http://www.ntru.com.

[18] A. Hülsing, J. Rijneveld,J. Schanck, J and P. Schwabe, High-Speed Key Encapsulation from NTRU. In: Fischer, W., Homma, N. (eds) Cryptographic Hardware and Embedded Systems – CHES 2017. CHES 2017. Lecture Notes in Computer Science(), vol. **10529**. Springer, Cham. https://doi.org/10.1007/978-3-319-66787-4_12

[19] P. Kirchner and P. A. Fouque, Revisiting Lattice Attacks on Overstretched NTRU Parameters. EUROCRYPT 2017, LNCS **10210**, Springer 2017, https://link.springer.com/chapter/10.1007/978-3-319-66787-4_12

[20] A. May, Cryptanalysis of NTRU (preprint), 1999.

[21] Robby G. McKilliam and Alex Grant and I. Vaughan L. Clarkson. Finding a closest point in a lattice of Voronoi's first kind 2014

[22] Micciancio, Daniele and Goldwasser, Shafi Complexity of Lattice Problems: A Cryptographic Perspective 2002

[23] Phong Q. Nguyen, Boosting the Hybrid Attack on NTRU: Torus LSH, Permuted HNF and Boxed Sphere, Computer Science, Mathematics, 2021.

[24] J. H. Silverman, Dimension-Reduced Lattices, Zero-Forced Lattices, and the NTRU Public Key Cryptosystem. Technical Report 13, Version 1, NTRU Cryptosystems, 1999.

[25] H. Silverman, N. P. Smart, and F. Vercauteren, An algebraic approach to NTRU ($q = 2n$) via Witt vectors and overdetermined systems of non linear equations. Security in Communication Networks – SCN 2004, LNCS **3352**, p. 278–298. Springer, 2005.

[26] Tsunekazu Saito, Keita Xagawa, and Takashi Yamakawa. Tightly-secure key-encapsulation mechanism in the quantum random oracle model. In Annual International Conference on the Theory and Applications of Cryptographic Techniques, pages 520–551. Springer, 2018.

[27] Selling, Eduard. "Ueber die binären und ternären quadratischen Formen." , vol. 1874, no. 77, 1874, pp. 143-229. https://doi.org/10.1515/crll.1874.77.143

Appendix A. Pseudocode for CVP in VFK lattices

Algorithm 2: Solving CVP in VFK lattices

input : $B \in \mathbb{N}^{m \times n+1}$ matrix with superbasis vectors $\mathbf{b}_i, i \in [1, n+1]$ as columns and $\mathbf{y} \in \mathbb{R}^{m \times 1}$ the target vector
output: A closest vector $\mathbf{v} \in \mathcal{L}$ of $\mathbf{y}$
$q_{ij} \leftarrow \mathbf{b}_i \mathbf{b}_j$; `/* for` $i, j = 1, ..., n+1, i \neq j$ `*/`
find z such that $\mathbf{y} = Bz$ $u \leftarrow \lfloor z \rfloor$;
for $i = 1$, ***to*** $n+1$ **do**
  $G \leftarrow flow_network(z, u, q)$; `/* see alg. 3 */`
  $t \leftarrow mincut(G)$; `/* see alg. 4 */`
  $u \leftarrow u + t$;
end

Algorithm 3: flow_network

input : target vector $z \in \mathbb{R}^{n+1 \times 1}$, $u \in \mathbb{N}^{n+1 \times 1}$, selling parameters q_{ij}
output: An undirected flow network G
$p \leftarrow z - u$;
Create Graph G with $n+3$ nodes and edges connecting them;
source $\leftarrow$ first node;
sink $\leftarrow$ last node;
Edges e_{ij} have the following weights:
for $i = 1$, ***to*** $n+1$ **do**
  for $j = i+1$, ***to*** $n+1$ **do**
    $w_{ij} = -q_{ij}$;
  end
  $s_i = -2 \sum_{j=1}^{n+1} q_{ij} p_j$;
  if $s_i \geq 0$ **then**
    $w_{i,sink} \leftarrow s_i$;
    $w_{source,i} \leftarrow 0$;
  else
    $w_{i,sink} \leftarrow 0$;
    $w_{source,i} \leftarrow -s_i$;
  end
end
return G

The minimum cut algorithm of 4, uses the Ford-Fulkerson procedure [12] for undirected graphs, that finds the max-flow of a graph, which is equivalent to the minimum cut [11]. $f(i,j), i, j \in E(G)$ describes the flow from the node i to node j.

In our experiments we are using the implimentation of networkx's mincut algorithm [3].

Algorithm 4: mincut

input : Graph G, source node s, sink node t
output: vector $t \in \{0,1\}^{n+1}$
for *each edge* $(u,v) \in E(G)$ **do**
  $f(u,v) \leftarrow 0$;
  $f(v,u) \leftarrow 0$;
end
while $\exists$ *a path* p *from* s *to* t *with no cycles in the residual network* G_f **do**
  $c_f(p) \leftarrow min\{c_f(u,v) : (u,v) \in p\}$; `/* residual capacity */`
  for *each edge* $(u,v) \in p$ **do**
    $f(u,v) \leftarrow f(u,v) + c_f(p)$;
    $f(v,u) \leftarrow -f(u,v)$;
  end
end
$C \leftarrow$ all the vertices that exist on the max-flow path;
for $i = 1$, ***to*** $n+1$ **do**
  if $v_i \in C$ **then**
    $t_i = 1$;
  else
    $t_i = 0$;
  end
end
return t

Appendix B. Babai vs CVP on VFK

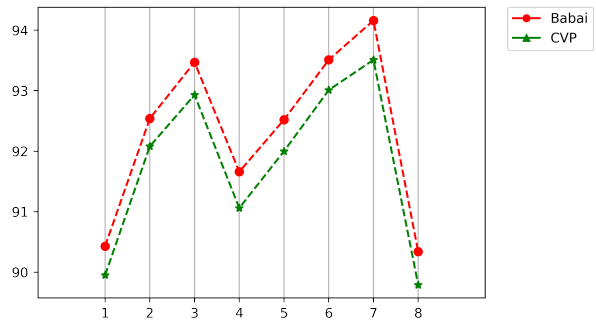


Figure 1. We use the matrix (6.2) with $N = 677$ and parameters $(P, q, k) = (6, 70, 10)$. We execute Babai and CVP (using the min-cut algorithm) with various target vectors of the form $\mathbf{t} \xleftarrow{\$} \{0,1\}^N \times \{-1000, ..., 1000\}^N$. The $y-$axis contains $||\mathbf{x} - \mathbf{t}||$, where $\mathbf{x}$ is either the output from CVP or Babai.

[3] `https://networkx.org/`

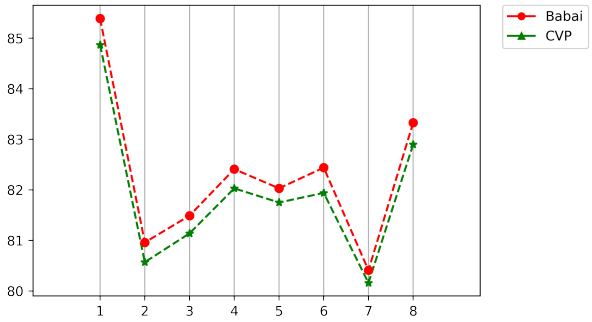


FIGURE 2. We fix the matrix (6.2) with $N = 557$ and parameters $(P, q, k) = (6, 70, 10)$. We execute Babai and CVP (using the min-cut algorithm) with various target vectors of the form $\mathbf{t} \xleftarrow{\$} \{0,1\}^N \times \{-1000, ..., 1000\}^N$. The $y-$axis contains $||\mathbf{x} - \mathbf{t}||$, where $\mathbf{x}$ is either the output from CVP or Babai. The $x-$axis counts the number of experiments.

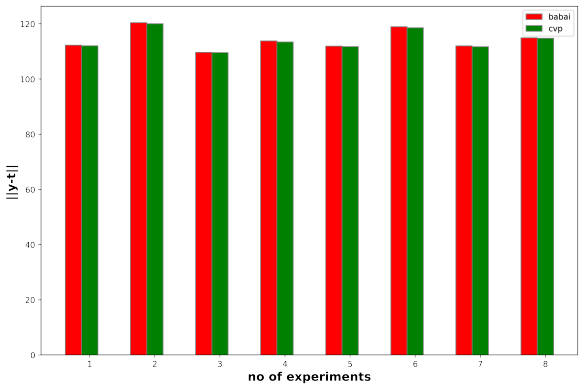


FIGURE 3. We fix the matrix (6.2) with $N = 239$ and parameters $(P, q, k) = (11, 256, 23)$. We execute Babai and CVP (using the min-cut algorithm) with various target vectors of the form $\mathbf{t} \xleftarrow{\$} \{0,1\}^N \times \{-1000, ..., 1000\}^N$. The $y-$axis contains $||\mathbf{y} - \mathbf{t}||$, where $\mathbf{y}$ is either the output from CVP or Babai. The $x-$axis counts the number of experiments.

In all the experiments Babai's algorithm provided a larger vector. However, the distance between the two vectors, i.e. the one provided by CVP and the other provided by Babai, are very close.